

ERCO ENERGÍA · PLATAFORMA ENRG · PARTE B

Diseño de Arquitectura Datalake + Sistema IA

Documento técnico — Prueba Técnica Líder de Datos e Inteligencia Artificial

Stack de referencia

AWS · dbt · LangGraph

Fuentes de datos

Telemedida · Clientes · Documentos

Agente

LUMI sobre capa Gold

Roadmap

3 personas · 6 meses

SECCIÓN 1 DE 4

Datalake — Arquitectura por capas en AWS

El datalake consolida tres tipos de fuentes con naturalezas muy distintas: **telemedida horaria** (alta frecuencia, ruidosa, de campo), **contexto de clientes** (baja frecuencia, semiestructurado) y **documentos no estructurados** (datasheets de máquinas, manuales técnicos, contratos). La separación en capas permite que cada fuente avance a su propio ritmo de calidad sin bloquear a las demás.



Fig. 1 – Arquitectura datalake por capas. Las flechas verdes indican la transición hacia gold, donde los datos son consumibles por el agente y la app.

LANDING	BRONZE	SILVER	GOLD
→ S3 bucket raw → Trigger S3 event → Sin transformación → Retención: 90 días → Partición: fuente/ fecha	→ Parquet sin pérdida → +_ingested_at, _hash → +_row_number (linaje) → Glue Job (Spark) → Catálogo automático	→ Reglas de calidad → Quarantine separado → dbt tests como contrato → Athena-queryable → Alertas SNS si falla	→ Modelos dbt agregados → Join tele + contexto → Embeddings documentos → Vistas por cliente_id → Base del agente LUMI

Decisiones de arquitectura y trade-offs

Decisión	Opción elegida	Alternativa descartada	Razón
Formato de almacenamiento	Parquet + S3	CSV comprimido	Columnar: Athena lee solo las columnas necesarias, 10× más barato en queries analíticas
Orquestación	Step Functions	Airflow en EC2	Serverless, nativo AWS, reintentos y ramas condicionales sin infraestructura que mantener
Transformaciones	dbt + Athena	PySpark puro	dbt tests son el contrato de calidad; SQL es legible por el equipo de negocio
Documentos no estructurados	S3 + OpenSearch	Solo S3 con Lambda	OpenSearch permite búsqueda semántica con embeddings; escala sin rediseñar el pipeline

Gobierno de accesos

IAM + Lake
Formation

ACLs manuales
en S3

Lake Formation permite control a nivel de
columna y fila — crítico para datos de energía
de múltiples clientes

Gobierno: linaje, catálogo y calidad

Cada registro lleva tres campos de linaje desde bronce: `_source_hash` (identifica el archivo de origen), `_ingested_at` (timestamp UTC de ingesta) y `_row_number` (posición en el CSV original). Esto permite rastrear cualquier cifra en gold hasta su registro crudo de campo.

El catálogo vive en **AWS Glue Data Catalog**. Los dbt tests actúan como contrato de calidad: si la tasa de nulos en `timestamp` supera el 5% o aparecen valores negativos sin flag, el pipeline falla y SNS notifica al equipo antes de que los datos lleguen a gold.

SECCIÓN 2 DE 4

Sistema IA — Arquitectura del agente LUMI

LUMI combina dos tipos de conocimiento: **datos numéricos precisos** del datalake (consumo, tendencias, anomalías) y **contexto no estructurado** del cliente (datasheets de máquinas, manuales, historial de incidencias). La arquitectura garantiza que el LLM nunca calcule cifras — solo interprete resultados ya computados.

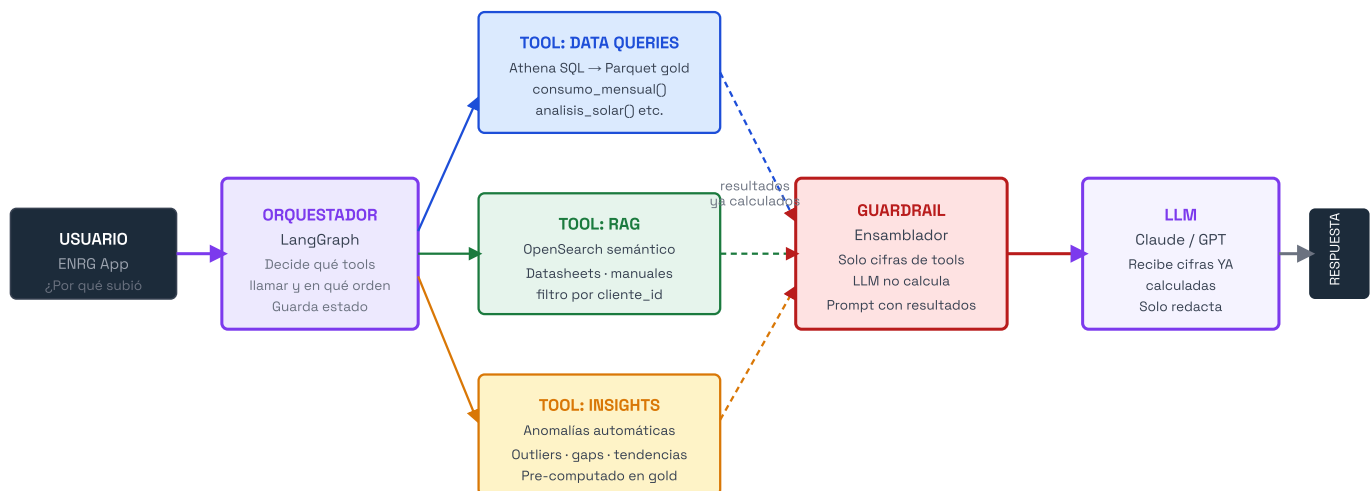


Fig. 2 — Arquitectura del agente LUMI. El guardrail garantiza que el LLM reciba cifras pre-computadas por los tools, nunca las calcule por su cuenta.

Cómo evitar respuestas inventadas sobre cifras

Este es el riesgo central de cualquier agente sobre datos numéricos. La solución tiene tres capas:

- 1. Separación estructural:** el LLM y las funciones de cálculo son componentes distintos. El LLM nunca tiene acceso directo a los datos — solo recibe strings con resultados ya calculados por el data layer (Athena/Parquet).
- 2. System prompt restrictivo:** el prompt prohíbe explícitamente inventar o inferir cifras. Si el contexto entregado no contiene el número que el usuario pide, el agente responde "No tengo ese dato disponible."
- 3. Trazabilidad en respuesta:** cada cifra en la respuesta puede rastrearse a la función que la generó. En producción, las respuestas incluyen una nota de fuente (*"según datos silver al 30/06/2026"*) que permite auditar.

Ejemplo de extremo a extremo

Pregunta: "¿Por qué subió mi factura en junio?" — CLI-001 Textiles del Valle

- 1 Router (LangGraph)** recibe la pregunta y determina que necesita: consumo mensual, desglose por franja horaria, y contexto del cliente. Athena
- 2 Tool: data_queries** ejecuta `consumo_mensual("CLI-001")` → mayo: 283,556 kWh / junio: 318,857 kWh / delta: +12.4%. Ejecuta `comparacion_mayo_junio_detalle()` → franja noche subió +36,095 kWh. SQL → Parquet
- 3 Tool: RAG** busca en OpenSearch documentos de CLI-001 con query "turno nocturno compresores" → recupera un fragmento del datasheet del compresor tornillo 75 kW que menciona el consumo en arranque.
RAG filtrado por cliente_id
- 4 Guardrail / Ensamblador** construye el system prompt con todas las cifras ya calculadas. El LLM recibe: "consumo mayo = 283,556 kWh, junio = 318,857 kWh, delta = +12.4%, aumento concentrado en franja noche 18-24h: +36,095 kWh..." LLM no calcula
- 5 LLM** redacta la respuesta usando solo las cifras entregadas: interpreta el patrón, cruza con el contexto de la textilera y el datasheet del compresor, y da una hipótesis de causa con recomendación. Solo redacción

SECCIÓN 3 DE 4

Operación — Calidad, costos y seguridad

Evaluación de calidad del agente

Dimensión	Métrica	Implementación
-----------	---------	----------------

Cifras correctas	% de cifras en respuesta que coinciden con la consulta fuente	Test automatizado: extraer números de respuesta y comparar con resultado del tool
Alucinación	% de respuestas con cifras no presentes en el contexto entregado	Regex sobre respuesta vs dict de resultados; umbral: 0 tolerancia
Relevancia	RAGAS faithfulness + answer relevancy	Evaluar muestra semanal con LLM-as-judge sobre dataset de preguntas conocidas
Latencia	p95 tiempo de respuesta	CloudWatch metric; alerta si p95 > 8s
Cobertura de tools	% preguntas que lograron invocar al menos 1 tool	Log de cada ejecución en DynamoDB; alerta si baja del 90%

Control de costos de inferencia

El riesgo principal: un agente que llame al LLM para cada consulta a +200 clientes × 24 horas puede volverse inviable rápidamente. Tres controles:

Caché de contexto: el resumen ejecutivo del cliente se calcula una vez al día y se cachea en ElastiCache. Las preguntas recurrentes del mismo cliente en el mismo día reutilizan el contexto sin re-ejecutar las queries de Athena.

Modelo por tarea: el router usa un modelo pequeño (Haiku / GPT-3.5) para clasificar la intención. Solo cuando la pregunta requiere síntesis compleja se invoca el modelo grande (Sonnet / GPT-4). El 70% de las preguntas típicas se resuelven con el modelo pequeño.

Presupuesto por cliente: cada cliente tiene un cupo mensual de tokens configurable. Si se supera, el agente responde con las métricas precalculadas en gold sin llamar al LLM.

Seguridad — qué datos ve el agente y de qué cliente

AISLAMIENTO POR CLIENTE_ID

- ✓ Todas las queries del data layer incluyen `WHERE cliente_id = ?` como parámetro obligatorio — nunca como opción
- ✓ El RAG filtra por `cliente_id` antes de hacer el vector search — un cliente no puede ver documentos de otro
- ✓ AWS Lake Formation controla acceso a nivel de fila en las tablas gold
- ✓ El agente no recibe el contexto de otros clientes en ningún paso del flujo

LIMITACIONES CONOCIDAS Y MITIGACIÓN

- ⚠ El historial de conversación en memoria podría filtrar datos entre sesiones → se limpia al cerrar sesión
- ⚠ El LLM podría memorizar patrones de clientes en fine-tuning → no se hace fine-tuning con datos de clientes
- ⚠ Logs de CloudWatch contienen queries con datos → acceso restringido a equipo de ops con MFA
- ⚠ Auditoría: cada llamada al agente se registra con `cliente_id`, pregunta y tools invocados

SECCIÓN 4 DE 4

Roadmap — 3 personas • 6 meses

Con 3 personas el criterio de priorización es uno: **¿qué desbloquea lo siguiente?** Sin datos confiables no hay agente. Sin agente no hay valor diferencial. El lujo (insights proactivos, fine-tuning, multi-idioma) va al final.

FASE 1 · Meses 1-2

Datos confiables en producción

- Pipeline bronze → silver en AWS (Glue + Step Functions)
- Catálogo Glue + dbt tests como contrato
- Quarantine con alertas SNS
- Ingesta de contexto de clientes (CRM)
- Dashboards básicos en ENRG (Athena → QuickSight)
- Gestión de accesos (Lake Formation)

Sin esto no hay nada. Un agente sobre datos sucios da respuestas equivocadas — peor que no tener agente.

FASE 2 · Meses 3-4

Gold layer + LUMI v1

- Modelos dbt en gold (consumo diario, mensual, benchmarks)
- Data access layer (funciones de consulta)
- LUMI v1: responde las 5 preguntas más frecuentes
- Ingesta y chunking de documentos (datasheets)
- RAG con OpenSearch, filtro por cliente_id
- API REST del agente (Lambda + API Gateway)

El agente sin gold layer confiable no puede responder con cifras reales. Primero los modelos, luego el chat.

FASE 3 · Meses 5-6

Operación + motor de insights

- Evaluación automática del agente (RAGAS)
- Motor de insights proactivos (anomalías sin pregunta)
- Control de costos (caché + modelo routing)
- Cobertura completa de fuentes (más clientes)
- Reportes automáticos mensuales por cliente
- Feedback loop: usuarios marcan respuestas incorrectas

Con el agente funcionando, el foco pasa a confiabilidad y escala. Los insights proactivos son el diferencial competitivo de ENRG.